98.6
Pass

Anand Shekhar

PDF generated at: 26 Feb 2026 06:37:01 UTC

View this report on HackerRank

Score

98.6% • 172.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 53 min 37 sec on 26 Feb 2026 11:11:15 IST

Candidate Information

Email	anandshekhar9560@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	26 Feb 2026 11:11:15 IST
Time taken	53 min 37 sec/ 90 min
Work Experience	> 5 years
Invited by	Vipul
Invited on	24 Feb 2026 15:37:44 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	87%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	90%	Hard	100%
Spring	75%	JPA	50%
Spring Data	50%	Transactional	50%
Medium	98%	Application Properties Annotations	100%
Exception Handling	100%	Aspect-Oriented Programming	100%
Sorting	100%	Greedy Algorithms	100%
Two Pointers	100%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%		

Questions

Spring Boot (Advanced) • 22.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	39 sec	
2	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	1 min 36 sec	
3	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	5/5	-	28 sec	
4	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	25 sec	
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 57 sec	

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Maximum Array Correlation Coding	Problem Solving (Intermediate)	75/75	-	38 min 29 sec	

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	9 min 31 sec	✓

1. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

2. Spring Data Transactional

Partially correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
   @Id
   private Long id;
   private String name;
   private Integer courseLength;
   private Integer courseLevel;
   @JsonBackReference
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "student_id")
   private Student student;
   private boolean inProgress;
   //getter & setters
}
```

Student.java

```
@Entity
public class Student {
   @Id
   private Long id;
   private String name;
   private String surname;
   @JsonManagedReference
   @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
   private List<Lesson> lessons;
   //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
```

```
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** It will try to delete the student then the DataIntegrityViolationException will be thrown. **

- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.

- [ ] It will throw a RuntimeException with the message "Student not found".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

3. Configuration Properties

Correct

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }
```

```
  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- ○ The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- ● ** A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final. ** ✓

An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.

A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

4. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

private void manage(HttpSecurity http) {   http.csrf().allowed() }

private void manage(HttpSecurity http) {   http.csrf() }

private void manage(HttpSecurity http) {   http.csrf().enable() }

** None of the above. CSRF is configured by default when Spring Security Dependency is added. **

No comments.

5. Predict the Output

Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}
```

```
@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
      System.out.println("After method:" + joinPoint.getSignature());
      System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Ravi , second name - Shekhar and id - 123   **

- [ ] After method : com.test.model.Employee addEmployee(string,string,string) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(String,String,String)
Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Maximum Array Correlation

Correct

Coding | Sorting | Greedy Algorithms | Medium | Two Pointers

Question description

For two arrays *a* and *b* of equal length, the array correlation is defined as the sum of all values *b[i]* where *b[i]* is greater than *a[i]*.

Given two integer arrays *a* and *b* of the same length, rearrange the elements of array *b* to maximize the array correlation. Return the maximum possible array correlation value.

Note: You are not allowed to rearrange the elements of the array *a*.

Example:
n = 5, *a* = [1, 2, 3, 4, 5], *b* = [3, 5, 4, 6, 2]

b can be rearranged to [2, 3, 4, 5, 6]. Now, for each index, the integers in *b* are greater than those in *a.* The array correlation is 2 + 3 + 4 + 5 + 6 = 20.

Function Description
Complete the function *getMaxArrayCorrelation* in the editor with the following parameter(s):
a[n]: the fixed order array
b[n]: the array to reorder

Returns:

long int: the maximum possible array correlation

Constraints

- $1 \le n \le 2 \times 10^5$
- $1 \le a[i], b[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, that represents the number of elements in *a*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *a[i].*
The next line contains an integer, *n*, that represents the number of elements in *b*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *b[i].*

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     -----------
5       →  a[] size n = 5
1       →  a = [1, 4, 2, 1, 3]
4
2
1
3
5       →  b[] size n = 5
2       →  b = [2, 3, 1, 2, 2]
3
1
2
2
```

Sample Output

```
7
```

Explanation

The second array can be rearranged to *b = [2, 1, 3, 2, 2]*. *b[i] > a[i]* for *i* = 0, 3, and 4, and the array correlation is 2 + 3 + 2 = 7.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     -----------
4       →  a[] size n = 4
1       →  a = [1, 9, 4, 2]
9
4
2
4       →  b[] size n = 4
8       →  b = [8, 4, 3, 1]
4
3
1
```

Sample Output

```
15
```

Explanation

The second array can be rearranged to *b = [3, 1, 8, 4]*. *b[i] > a[i]* for *i =* 0, 3, and, 4. The array correlation is 3 + 8 + 4 = 15.

Interviewer guidelines

▼ SOLUTION

Skills: Greedy Algorithms, Sorting

Optimal Solution:

We can use a greedy approach here. We will always try to take the maximum values of b as much as possible. We can sort the two arrays in reverse order and iterate through the array *a.* We add *b[j]* to the score and increment *j* at each element where *a[i] < b[j].*

Python Solution:

```
def getMaxArrayCorrelation(a, b):
    list.sort(a, reverse = True)
    list.sort(b, reverse = True)
    ans = 0
    i, j = 0, 0
    n = len(a)
    while i < n and j < n:
        if a[i] < b[j]:
```

```
        ans += b[j]
        j += 1
    i += 1
return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n logn)

Sorting the arrays takes O(n logn) time in the worst case.

Space Complexity - O(n)

Assuming that the inbuilt sorting functions use a variation of merge sort, sorting uses O(n) extra space.

Candidate's Solution

Language used: **Java 8**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMaxArrayCorrelation' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY a
     *  2. INTEGER_ARRAY b
     */

    public static long getMaxArrayCorrelation(List<Integer> a, List<Integer> b) {
```

```java

        // int n = a.size();

         List<Integer> sortedA = new ArrayList<>(a);
         List<Integer> sortedB = new ArrayList<>(b);

         Collections.sort(sortedA);
         Collections.sort(sortedB);

         long result = 0;
         int i =sortedA.size() - 1;
         int j =sortedB.size() - 1 ;

         while(i >= 0 && j >= 0){
             if(sortedB.get(j)> sortedA.get(i)){
                 result += sortedB.get(j);
                 i--;
                 j--;
             }
             else{
                 i--;
             }

         }
         return result;
     }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int aCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> a = IntStream.range(0, aCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
```

```
            .collect(toList());

        int bCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> b = IntStream.range(0, bCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.getMaxArrayCorrelation(a, b);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	0.1213 sec	30.1 KB
Testcase 1	Easy	Sample	Success	1	0.1076 sec	30.1 KB
Testcase 2	Easy	Sample	Success	1	0.1036 sec	30.6 KB
Testcase 3	Easy	Hidden	Success	3	0.0958 sec	30.3 KB
Testcase 4	Easy	Hidden	Success	3	0.106 sec	30.5 KB

Testcase 5	Medium	Hidden	Success	4	0.1125 sec	30.1 KB
Testcase 6	Medium	Hidden	Success	5	0.1183 sec	31.4 KB
Testcase 7	Medium	Hidden	Success	6	0.1229 sec	31.9 KB
Testcase 8	Medium	Hidden	Success	6	0.1435 sec	32.3 KB
Testcase 9	Hard	Hidden	Success	7	0.2695 sec	45.1 KB
Testcase 10	Hard	Hidden	Success	7	0.3112 sec	55 KB
Testcase 11	Hard	Hidden	Success	7	0.4128 sec	55.8 KB
Testcase 12	Hard	Hidden	Success	7	0.4992 sec	75.5 KB
Testcase 13	Hard	Hidden	Success	8	0.7353 sec	117 KB
Testcase 14	Hard	Hidden	Success	9	0.5083 sec	116 KB

No comments.

7. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number
3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- $1 \leq$ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ 10

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
   public void rob(IActions actions) throws InterruptedException, ExecutionException{
      ExecutorService threadPool = Executors.newFixedThreadPool(2);
      String passcodeOfDoor = actions.unlockTheDoor();
      Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
      Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));

      String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
      String safeDepositBoxPin = hackSecretPinTask.get();

      actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
      threadPool.shutdown();
   }
}
```

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
```

```java

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}

class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{

       String doorPasscode = actions.unlockTheDoor();
       ExecutorService executor = Executors.newFixedThreadPool(2);

       try{
          Future<String> pinFuture = executor.submit(() -> actions.hackSecretPin(doorPasscode));
          Future<String> boxFuture = executor.submit(() -> actions.figureOutSafeDepositBoxNumber(doorPasscode));

          String safeDepositBoxPin = pinFuture.get();
          String safeDepositBoxNumber = boxFuture.get();

          actions.openSafeDepositLock(safeDepositBoxNumber, safeDepositBoxPin);
       }
       finally{
        executor.shutdown();
       }
    }
}

public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

```

```java
class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
```

```
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.1122 sec	30.1 KB
Testcase 1	Hard	Hidden	Success	74	2.1061 sec	30.3 KB

No comments.